

# Sprint 7 Structured Notes: GitHub Repository and Remotes

## 1. Current Progress

You are now in **Sprint 7** of the Relational Databases learning path.

The full learning path is:

**Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance**

So far, you have already completed these stages:

### Sprint 1: Tools and VS Code

You learned how to prepare your coding environment. You learned about:

- VS Code
- Code editors and IDEs
- Workspaces
- Project folders
- `index.html`
- Live Server
- Basic VS Code shortcuts
- Beginner extensions

**Simple meaning:** Sprint 1 helped you prepare the place where you write and manage project files.

### Sprint 2: Terminal, Shell, and Command Line Basics

You learned what happens when you type commands. You learned about:

- Terminal
- Command line
- Shell
- Bash
- Command history
- Auto-complete
- Clearing the terminal screen
- Stopping a running command

**Simple meaning:** Sprint 2 helped you understand the environment where commands are typed and run.

### Sprint 3: Bash File and Folder Commands

You learned how to manage files and folders using Bash. You learned commands such as:

- `pwd`
- `cd`
- `ls`
- `mkdir`
- `touch`
- `echo`
- `cat`
- `less`
- `mv`
- `cp`
- `rm`
- `>`
- `>>`

**Simple meaning:** Sprint 3 taught you how to move around folders, create files, read files, rename files, copy files, delete files, and write text into files using commands.

### Sprint 4: Bash Scripting Basics

You learned how to save Bash commands inside a file and run them together. You learned about:

- Bash scripts
- `.sh` files
- Shebang
- Interpreter paths
- Variables
- Variable interpolation
- Arrays
- `for` loops
- `do` and `done`
- `echo` logging
- `rsync` idea
- Binaries

**Simple meaning:** Sprint 4 taught you how to turn Bash commands into a reusable script.

### Sprint 5: Nano and Git Setup

You learned two important things:

- How to edit files in the terminal using Nano
- How to prepare Git correctly before starting real Git work

You learned about:

- Nano basics
- Save and exit in Nano
- What Git is
- What version control means
- What a commit is
- What a branch is
- Git vs GitHub
- Checking Git installation
- Setting `user.name`
- Setting `user.email`
- Setting `core.editor`
- Viewing Git config

**Simple meaning:** Sprint 5 prepared your terminal editing skill and Git identity setup.

## Sprint 6: Local Git Workflow

You learned the first real Git workflow on your computer. You learned about:

- Local repository
- `git init`
- Hidden `.git` folder
- Working tree
- File states
- Untracked files
- Modified files
- Staged files
- Staging area
- `git status`
- `git add`
- `git reset`
- `git commit`
- Commit messages
- Commit hashes
- `git log`

**Simple meaning:** Sprint 6 taught you how to track project changes locally on your own computer.

## Sprint 7: GitHub Repository and Remotes

Now you are ready for the next step. In Sprint 6, your Git project stayed only on your own computer. In Sprint 7, you learn how to connect that local Git project to **GitHub**.

**Simple meaning:** Sprint 7 teaches you how to move from local Git work to online Git work.

## 2. Main Goal of Sprint 7

The main goal of Sprint 7 is:

### Connect a local project to GitHub.

By the end of Sprint 7, you should be able to:

- Understand local repository vs remote repository
- Create a GitHub repository
- Understand public vs private repositories
- Understand README, `.gitignore`, and license choices
- Understand HTTPS vs SSH
- Understand the idea of access tokens
- Understand the idea of SSH keys
- Understand what `origin` means
- Check remotes with `git remote -v`
- Clone a repository from GitHub
- Connect an existing local repository to GitHub
- Push local commits to GitHub
- Pull changes from GitHub
- Understand what `-u` means in `git push -u origin main`
- Understand the idea of GitHub CLI

**Simple meaning:** Sprint 7 teaches you how to connect your Git project on your computer to GitHub online.

---

## 3. Why Sprint 7 Is Important

Sprint 7 is important because modern Git work is usually not only local. Most developers use remote repositories so they can:

- Back up their work online
- Share projects with others
- Collaborate in teams
- Continue work on different computers
- Prepare for branching, pull requests, and code review later

If Sprint 6 taught you how to save project history on your computer, Sprint 7 teaches you how to store that history online.

**Simple meaning:** Sprint 7 is the bridge between local Git and online collaboration.

---

## 4. Big Beginner Idea

Think like this:

```
Your computer = local  
GitHub = remote / online
```

In Sprint 6:

```
Project → Git history on your computer
```

In Sprint 7:

```
Project → Git history on your computer → GitHub online copy
```

**Simple meaning:** GitHub becomes the online home of your Git project.

---

## 5. What Is a Local Repository?

A **local repository** is a Git project stored on your own computer.

Example:

```
sprint6-practice/
```

If that folder contains a hidden `.git` folder, it is a local Git repository.

**Simple meaning:** A local repository is a Git-tracked project on your computer.

---

## 6. What Is a Remote Repository?

A **remote repository** is a Git repository stored online. It is usually stored on a platform like GitHub or GitLab.

Example idea:

```
github.com/your-username/my-project
```

**Simple meaning:** A remote repository is the online version of your Git project.

---

## 7. Local vs Remote Repository

### Local repository

- Stored on your computer
- You work on files here directly
- Commits are first created here

### Remote repository

- Stored online
- Used for sharing, backup, and collaboration
- Can receive your commits through `git push`

**Simple meaning:** Local is on your computer. Remote is online.

---

## 8. Git vs GitHub Reminder

Many beginners confuse Git and GitHub. They are connected, but they are not the same.

| Tool   | Simple Meaning                                    |
|--------|---------------------------------------------------|
| Git    | Tool on your computer that tracks project changes |
| GitHub | Website that stores Git projects online           |
| GitLab | Another website that stores Git projects online   |

**Simple meaning:** Git tracks changes. GitHub stores Git repositories online.

---

## 9. What Is a GitHub Repository?

A GitHub repository is an online project folder stored on GitHub. It can contain files such as:

- `README.md`
- `index.html`
- CSS files
- JavaScript files
- SQL files
- Bash scripts
- Notes
- Documentation

**Simple meaning:** A GitHub repository is your project saved online.

---

## 10. Creating a GitHub Repository

When creating a repository on GitHub, you may see options such as:

- Owner
- Repository name
- Description
- Public or Private
- Add README
- Add `.gitignore`
- Add license
- Use a template

### Owner

This is the account or organization that owns the repository.

**Simple meaning:** Owner means who the repository belongs to.

### Repository name

This is the project name. Example:

```
sprint7-practice
```

**Simple meaning:** The repository name is the project folder name on GitHub.

### Description

This is a short explanation of the project.

**Simple meaning:** Description tells what the project is about.

### Template

Some repositories can be created from a template. A template is a pre-made project starting point.

**Simple meaning:** A template helps you start from an existing project structure.

---

## 11. Public vs Private Repository

A GitHub repository can be **public** or **private**.

### Public repository

A public repository can be seen by other people. Use public when:

- You want to share your work
- You want to build a portfolio
- You are doing open-source work

**Simple meaning:** Public means other people can see the repository.

### Private repository

A private repository can only be seen by you and people you allow. Use private when:

- The project is personal
- The work is not ready to share
- The project contains private work

**Simple meaning:** Private means only allowed people can see the repository.

**Beginner note:** For practice, private is often a safe choice unless your teacher asks for public.

---

## 12. What Is a README?

A `README.md` file explains the project. It often appears on the main page of a GitHub repository.

A README may include:

- Project name
- What the project does
- How to run it
- Important notes
- What you are learning

Example idea:

```
# Sprint 7 Practice

This is my GitHub remote practice project.

I am learning:
```



- local repository
- remote repository
- git push
- git pull
- git clone

**Simple meaning:** README is the front-page explanation of your project.

---

## 13. What Is .gitignore?

A `.gitignore` file tells Git which files or folders should not be tracked.

Example:

```
.env  
node_modules/  
dist/
```

**Simple meaning:** `.gitignore` tells Git, "Do not include these files in project history."

### Why `.gitignore` is important

Some files should not be pushed to GitHub, such as:

- Secret files
- Password files
- Environment files
- Dependency folders
- Build output folders
- IDE/system files

Important example:

```
.env
```

This file often contains secrets. Do not push it to GitHub.

---

## 14. What Is a License?

A license tells other people what they are allowed to do with your code.

A license answers questions such as:

- Can someone use this code?
- Can someone copy it?
- Can someone modify it?
- Can someone use it commercially?

**Simple meaning:** A license is the legal rule for using your project.

**Beginner note:** For now, you only need to understand what a license is.

---

## 15. What Is Clone?

The command `git clone` copies a remote repository from GitHub to your computer.

Example:

```
git clone git@github.com:your-username/your-repository-name
```

After cloning, you usually move into the folder:

```
cd name-of-cloned-repo
```

**Simple meaning:** Clone means download a GitHub repository to your computer.

---

## 16. What Is Push?

The command `git push` sends your local commits to the remote repository on GitHub.

Example:

```
git push -u origin main
```

**Simple meaning:** Push means upload your commits from your computer to GitHub.

In Sprint 6, commits stayed only on your computer. In Sprint 7, `git push` sends them online.

---

## 17. What Is Pull?

The command `git pull` brings changes from GitHub to your local computer.

Example:

```
git pull
```

Use pull when:

- You changed something on GitHub website
- Another person pushed changes
- You want your local project to become up to date

**Simple meaning:** Pull means download the latest GitHub changes into your local project.

---

## 18. Push vs Pull vs Clone

| Command                | Simple Meaning                          |
|------------------------|-----------------------------------------|
| <code>git clone</code> | Copy GitHub repository to your computer |
| <code>git push</code>  | Send local commits to GitHub            |
| <code>git pull</code>  | Bring GitHub changes to your computer   |

Simple memory:

```
clone = first full download  
push = upload your commits  
pull = download latest changes
```

## 19. HTTPS vs SSH

When you click the green **Code** button on GitHub, you usually see clone choices such as:

- HTTPS
- SSH
- GitHub CLI

## HTTPS

HTTPS uses a link like this:

```
https://github.com/username/repo.git
```

For private work or pushing changes, authentication is needed. GitHub usually uses access tokens instead of old-style password use.

**Simple meaning:** HTTPS can work, but it may ask for token-based authentication.

## SSH

SSH uses a link like this:

```
git@github.com:username/repo.git
```

SSH uses a public/private key pair. It is often more convenient after setup because you do not need to enter tokens again and again.

**Simple meaning:** SSH lets your computer prove securely that it is really you.

---

## 20. What Is an Access Token?

An **access token** is like a special password used for GitHub authentication. It is used especially with HTTPS workflows.

**Simple meaning:** Access token means a special GitHub password for Git operations.

### Important rules

- Do not share your token
  - Do not write it inside code
  - Do not push it to GitHub
- 

## 21. What Is an SSH Key?

An **SSH key** is used to connect your computer to GitHub securely. It has two parts:

- Private key
- Public key

## Private key

This stays on your computer.

**Simple meaning:** Private key is your secret key. Do not share it.

## Public key

This is added to GitHub.

**Simple meaning:** Public key is the key GitHub can use to recognize your computer.

**Important idea:**

- Private key stays local
  - Public key is shared for verification
- 

## 22. What Is `origin`?

`origin` is the default nickname for the main remote repository.

Example:

```
git remote add origin git@github.com:your-github-username/name-of-repo.git
```

Here:

- `origin` = nickname
- `git@github.com:...` = real repository address

**Simple meaning:** `origin` usually means your main GitHub remote repository.

---

## 23. What Is `git remote -v`?

This command shows which remote repository your local project is connected to.

```
git remote -v
```

Example output:

```
origin  git@github.com:maleesha/sprint7-practice.git (fetch)
origin  git@github.com:maleesha/sprint7-practice.git (push)
```

**Simple meaning:** `git remote -v` shows the GitHub address connected to your local project.

### What fetch and push mean here

- `fetch` = where Git reads changes from
- `push` = where Git sends changes to

**Simple meaning:** The output shows where your project gets changes from and where it sends changes to.

---

## 24. What Does `-u` Mean in `git push -u origin main`?

Command:

```
git push -u origin main
```

This means:

- Push the local `main` branch to `origin/main`
- Remember this connection for next time

The `-u` flag is short for `--set-upstream`. You usually use it once for a new branch. After that, future pushes can often be done with just:

```
git push
```

**Simple meaning:** `-u` connects the local branch and remote branch so future pushes are easier.

---

## 25. What Is GitHub CLI?

GitHub CLI is a command-line tool for GitHub. It uses the command:

```
gh
```

Examples:

```
gh repo clone your-username/your-repository-name
gh repo create
```

**Simple meaning:** GitHub CLI lets you create and manage GitHub repositories from the terminal.

**Beginner note:** The GitHub website is often easier at first, but GitHub CLI is good to know.

---

## 26. Sprint 7 Required Commands

These are the important Sprint 7 commands:

```
git clone git@github.com:your-username/your-repository-name
cd name-of-cloned-repo
git remote -v
gh repo clone your-username/your-repository-name
gh repo create
git remote add origin git@github.com:your-github-username/name-of-repo.git
git push -u origin main
git pull
```

## 27. Sprint 7 Workflow Option A: Create on GitHub First, Then Clone

Use this when you want to create the repository on GitHub first.

### Step 1: Create the repository on GitHub

Choose:

- Repository name
- Public or Private
- README if wanted
- `.gitignore` if wanted
- License if wanted

### Step 2: Copy the clone URL

Example SSH URL:

```
git@github.com:your-username/sprint7-practice.git
```

### Step 3: Clone the repository

```
git clone git@github.com:your-username/sprint7-practice.git
```

### Step 4: Enter the folder

```
cd sprint7-practice
```

### Step 5: Check the remote

```
git remote -v
```

### Step 6: Create or edit a file

Example:

```
echo "I am learning Sprint 7 GitHub remotes." >> README.md
```

### Step 7: Check status

```
git status
```

### Step 8: Stage the file

```
git add README.md
```

### Step 9: Commit

```
git commit -m "docs: update README for Sprint 7"
```

### Step 10: Push

```
git push
```

**Simple meaning:** You cloned a GitHub repository, changed a file, committed it, and pushed it back to GitHub.

---



## 28. Sprint 7 Workflow Option B: Start from an Existing Local Repository, Then Push to GitHub

Use this when you already have a local Git project from Sprint 6.

### Step 1: Go to your local repository

```
cd sprint6-practice
```

### Step 2: Check status

```
git status
```

Make sure your work is committed before connecting it to GitHub.

### Step 3: Create an empty repository on GitHub

If your local project already has files like `README.md`, do not add another README on GitHub during creation. This helps avoid beginner confusion.

### Step 4: Add the remote

```
git remote add origin git@github.com:your-github-username/name-of-repo.git
```

### Step 5: Check the remote

```
git remote -v
```

### Step 6: Push for the first time

```
git push -u origin main
```

**Simple meaning:** You connected your existing local project to GitHub and uploaded it.

---

## 29. Important Sprint 7 Concepts in Simple Words

### Local repository

Git project on your computer.

## **Remote repository**

Git project stored online.

## **GitHub repository**

Online project folder on GitHub.

## **Public repository**

Visible to others.

## **Private repository**

Visible only to allowed people.

## **README**

Project explanation file.

`.gitignore`

File that tells Git what not to track.

## **License**

Rule for how other people may use the code.

## **HTTPS**

Remote connection method that often uses token authentication.

## **SSH**

Remote connection method using keys.

## **Access token**

Special GitHub password for authentication.

## **SSH key**

Secure identity key used to connect your computer to GitHub.

`origin`

Default name for the main remote repository.

```
git remote -v
```

Shows connected remote repository addresses.

```
git clone
```

Copies a GitHub repository to your computer.

```
git push
```

Sends local commits to GitHub.

```
git pull
```

Brings GitHub changes to your computer.

```
-u
```

Sets the upstream connection between local and remote branch.

## GitHub CLI

Terminal tool for GitHub using `gh`.

---

## 30. Common Beginner Mistakes

### Mistake 1: Thinking GitHub automatically receives local commits

Wrong idea:

```
I committed, so GitHub already has my changes.
```

Correct idea:

```
Commit saves locally.  
Push sends to GitHub.
```

**Simple meaning:** Local commit is not the same as GitHub upload.

### Mistake 2: Forgetting to check remotes

Before pushing, check:

```
git remote -v
```

**Simple meaning:** Always confirm where your project is connected.

### Mistake 3: Creating README in both places

If your local project already has `README.md` and GitHub also creates a new README, beginners may run into history confusion.

**Beginner habit:** If your local project already has files, create an empty GitHub repository.

### Mistake 4: Using the wrong branch name

Some repositories use:

```
main
```

Older ones may use:

```
master
```

Check with:

```
git branch
```

**Simple meaning:** Push the correct branch name.

### Mistake 5: Sharing secrets

Never push:

- Passwords
- Tokens
- `.env` files
- Private keys

Use `.gitignore` when needed.

**Simple meaning:** Do not upload secrets to GitHub.

## Mistake 6: Confusing HTTPS and SSH

Both are connection methods, but they work differently.

**Simple meaning:** HTTPS often uses tokens. SSH uses keys.

## Mistake 7: Forgetting what `-u` does

Some beginners use `git push -u origin main` without understanding it.

**Simple meaning:** `-u` makes future pushes easier by remembering the branch connection.

---

## 31. Sprint 7 Checkpoint

Your Sprint 7 checkpoint is:

**Push a local project to GitHub and then clone it into another folder.**

That means you should be able to do this full flow:

```
git remote add origin git@github.com:your-username/sprint7-practice.git
git push -u origin main
cd ..
git clone git@github.com:your-username/sprint7-practice.git
cd sprint7-practice
git remote -v
```

If this works, you passed Sprint 7.

**Simple meaning:** This checkpoint proves that you can connect local Git and GitHub correctly.

---

## 32. Sprint 7 Command Summary

| Command                               | Simple Meaning                             |
|---------------------------------------|--------------------------------------------|
| <code>git clone &lt;url&gt;</code>    | Copy remote repository to your computer    |
| <code>cd name-of-cloned-repo</code>   | Enter the cloned project folder            |
| <code>git remote -v</code>            | Show connected remote repository addresses |
| <code>gh repo clone owner/repo</code> | Clone using GitHub CLI                     |
| <code>gh repo create</code>           | Create a GitHub repository from terminal   |

| Command                                        | Simple Meaning                                 |
|------------------------------------------------|------------------------------------------------|
| <code>git remote add origin &lt;url&gt;</code> | Connect local project to remote repository     |
| <code>git push -u origin main</code>           | Push first time and remember branch connection |
| <code>git push</code>                          | Send commits to GitHub                         |
| <code>git pull</code>                          | Bring latest remote changes to your computer   |

### 33. Sprint 7 Key Terms Summary

| Term                                          | Simple Meaning                                      |
|-----------------------------------------------|-----------------------------------------------------|
| Local repository                              | Git project on your computer                        |
| Remote repository                             | Git project stored online                           |
| GitHub repository                             | Online project folder on GitHub                     |
| Public repository                             | Repository visible to others                        |
| Private repository                            | Repository visible only to allowed people           |
| README                                        | Project explanation file                            |
| <code>.gitignore</code>                       | File that tells Git what not to track               |
| License                                       | Rule for how others may use the code                |
| HTTPS                                         | Remote connection method often using tokens         |
| SSH                                           | Remote connection method using keys                 |
| Access token                                  | Special GitHub password for authentication          |
| SSH key                                       | Secure key for connecting your computer to GitHub   |
| <code>origin</code>                           | Default remote name                                 |
| <code>git remote -v</code>                    | Show remote connections                             |
| <code>git clone</code>                        | Copy repository from GitHub to computer             |
| <code>git push</code>                         | Upload local commits to GitHub                      |
| <code>git pull</code>                         | Download latest GitHub changes                      |
| <code>-u</code> / <code>--set-upstream</code> | Remember connection between local and remote branch |
| GitHub CLI                                    | GitHub terminal tool using <code>gh</code>          |

## 34. Final Sprint 7 Understanding

After Sprint 7, you should understand that Git work is not only local. You should now understand:

- What a remote repository is
- The difference between local and remote
- What a GitHub repository is
- Public vs private repository
- Why README matters
- Why `.gitignore` matters
- What a license is
- The difference between HTTPS and SSH
- What an access token is
- What an SSH key is
- What `origin` means
- What `git remote -v` does
- What `git clone` does
- What `git push` does
- What `git pull` does
- What `git push -u origin main` means
- What GitHub CLI is
- How to connect an existing local repo to GitHub
- How to clone from GitHub to your computer

**Simple final meaning:** Sprint 7 teaches you how to connect your local Git project to GitHub so your project can live both on your computer and online.

---

## 35. Sprint 7 Mini Assessment

Answer these before moving to Sprint 8:

1. What is a remote repository?
2. What is the difference between local and remote?
3. What is GitHub used for?
4. What is a GitHub repository?
5. What is the difference between public and private repository?
6. What is a README?
7. What is `.gitignore`?
8. What is a license?
9. What is the difference between HTTPS and SSH?
10. What is an access token?
11. What is an SSH key?
12. What does `origin` mean?
13. What does `git remote -v` show?
14. What does `git clone` do?

15. What does `git push` do?
  16. What does `git pull` do?
  17. What does `git push -u origin main` mean?
  18. What is GitHub CLI?
  19. Why should you not push passwords or `.env` files to GitHub?
  20. What is the Sprint 7 checkpoint?
- 

## 36. Sprint 7 Completion Checklist

Tick these when completed:

- ☐ I can explain what a remote repository is.
  - ☐ I can explain the difference between local and remote.
  - ☐ I can create a GitHub repository.
  - ☐ I understand public vs private repositories.
  - ☐ I understand what README is.
  - ☐ I understand what `.gitignore` is.
  - ☐ I understand what a license is.
  - ☐ I understand HTTPS vs SSH.
  - ☐ I understand the idea of access tokens.
  - ☐ I understand the idea of SSH keys.
  - ☐ I know what `origin` means.
  - ☐ I can check remotes using `git remote -v`.
  - ☐ I can clone a repository.
  - ☐ I can connect a local repo to GitHub.
  - ☐ I can push local changes to GitHub.
  - ☐ I can pull remote changes from GitHub.
  - ☐ I understand what `-u` means in `git push -u origin main`.
  - ☐ I understand the idea of GitHub CLI.
  - ☐ I can complete the Sprint 7 checkpoint.
- 

## 37. One-Line Memory Rule

**Sprint 7 = connect your local Git project to GitHub using remotes, clone, push, and pull.**